



EduCredentials

Decentralised Education Credentials on Cardano

Full Technical Documentation

Milestone 16 — Cardano Starter Kit

Zonke / Tobb Technologies Programme

Author: Prefina Kayembe

Date: May 2026

A production-grade starter kit for issuing and verifying academic diplomas, certificates, and micro-credentials using Cardano blockchain technology. Built with Haskell / Plutus V2 on-chain, Node.js + MySQL off-chain, and a three-role HTML/CSS/JavaScript web frontend.

Table of Contents

1.	Project Introduction	3
2.	System Overview	4
3.	Architecture Documentation	5
4.	Blockchain Documentation	7
5.	Database Schema	9
6.	API Documentation	10
7.	User Roles and User Flows	12
8.	Frontend Architecture	13
9.	Setup and Deployment Instructions	14
10.	Privacy and Security Strategy	16
11.	Challenges and Limitations	17
12.	Future Improvements	18
13.	Conclusion	19

1 PROJECT INTRODUCTION

1.1 Purpose of the Project

EduCredentials is a full-stack decentralised application that enables educational institutions to issue tamper-proof academic credentials on the Cardano blockchain. The system replaces paper and PDF certificates with cryptographically verifiable digital credentials that can be checked by anyone, anywhere, instantly, without contacting the issuing institution.

The project was built as Milestone 16 of the Zonke / Tobb Technologies Programme, which requires a real Cardano starter kit combining a Plutus smart contract, a web application backend, and a professional frontend — all working together to demonstrate a meaningful real-world Cardano use case.

1.2 Problem Statement

Academic fraud is a global problem. Employers and institutions receive thousands of CV submissions with fake or exaggerated qualifications. Traditional verification requires the employer to contact the university directly, which is slow, expensive, and often impossible across international borders. In many countries, educational institutions have no centralised verification system at all.

The result is that employers either skip verification entirely, accept documents at face value, or spend significant resources on manual checks. This creates risk for employers and erodes trust in legitimate qualifications.

1.3 Why Cardano Blockchain

Cardano was chosen for several specific technical reasons. Its eUTxO model is well-suited to the NFT-style credential token design because each credential can be represented as a unique token with a deterministic identifier. The Plutus smart contract language, based on Haskell, provides formal verification properties that are important for a system handling academic records.

Cardano native tokens do not require a custom smart contract to exist — the minting policy controls who can create them. This means the on-chain footprint is minimal: one minting transaction per credential, with the credential hash as the token name. No gas-heavy smart contract execution is required for verification, which keeps costs low and verification fast.

The public nature of the Cardano blockchain means that any employer, regardless of their technical knowledge, can independently confirm a credential exists on-chain using a block explorer without needing access to any proprietary system.

1.4 Scope

In scope:

- Issuing diplomas, certificates, and micro-credentials through a web dashboard
- Anchoring a cryptographic hash of each credential on the Cardano blockchain
- Student credential claiming via a one-time secure link
- Instant public verification by credential hash or QR code
- Credential revocation by the issuing institution
- JWT-authenticated issuer access control
- Privacy-preserving design with zero PII on-chain
- Complete documentation and unit test suite

Out of scope for this starter kit:

- Live Cardano testnet deployment (contract written; deployment guide provided)
- Student wallet-based self-sovereign identity (SSI)
- GDPR right-to-erasure workflows
- Multi-institution federation
- Mobile native application

2 SYSTEM OVERVIEW

2.1 How the System Works

The system consists of four interconnected layers. At the top, a browser-based frontend provides role-specific dashboards for issuers, students, and verifiers. Beneath that, a Node.js backend API handles business logic, authentication, and database operations. A MySQL database stores all private off-chain data. Finally, the Cardano blockchain stores a tamper-proof hash of each credential as a native token.

2.2 The Core Concept — Hash Anchoring

The fundamental idea is that you do not need to store a credential on the blockchain to prove it is real. You only need to store a fingerprint. That fingerprint is the SHA-256 hash of the credential data. If anyone tampers with any field of the credential — the student name, the grade, the institution — the hash changes completely. The blockchain record no longer matches, and verification fails.

Example: The hash of "Bachelor of Science | University of Cape Town | Grade: First Class" is a1f3c8d2e9b7... — completely different from the hash of "Bachelor of Science | University of Cape Town | Grade: Pass". You cannot change any detail without the hash changing.

2.3 End-to-End Flow

Step	Actor	Action	Result
1	Issuer	Fills credential form in web dashboard	Credential saved in MySQL
2	Backend	Generates SHA-256 hash of canonical fields	28-byte credential hash created
3	Backend	Builds Cardano minting transaction	Unsigned tx body prepared
4	Issuer wallet	Signs and submits transaction	Hash minted as Cardano native token
5	System	Generates QR code and claim link	Sent to student by email
6	Student	Opens claim link in browser	Views and saves credential details
7	Verifier	Pastes hash on verifier page	Instant valid/invalid result displayed

3

ARCHITECTURE DOCUMENTATION

3.1 Full System Architecture Diagram

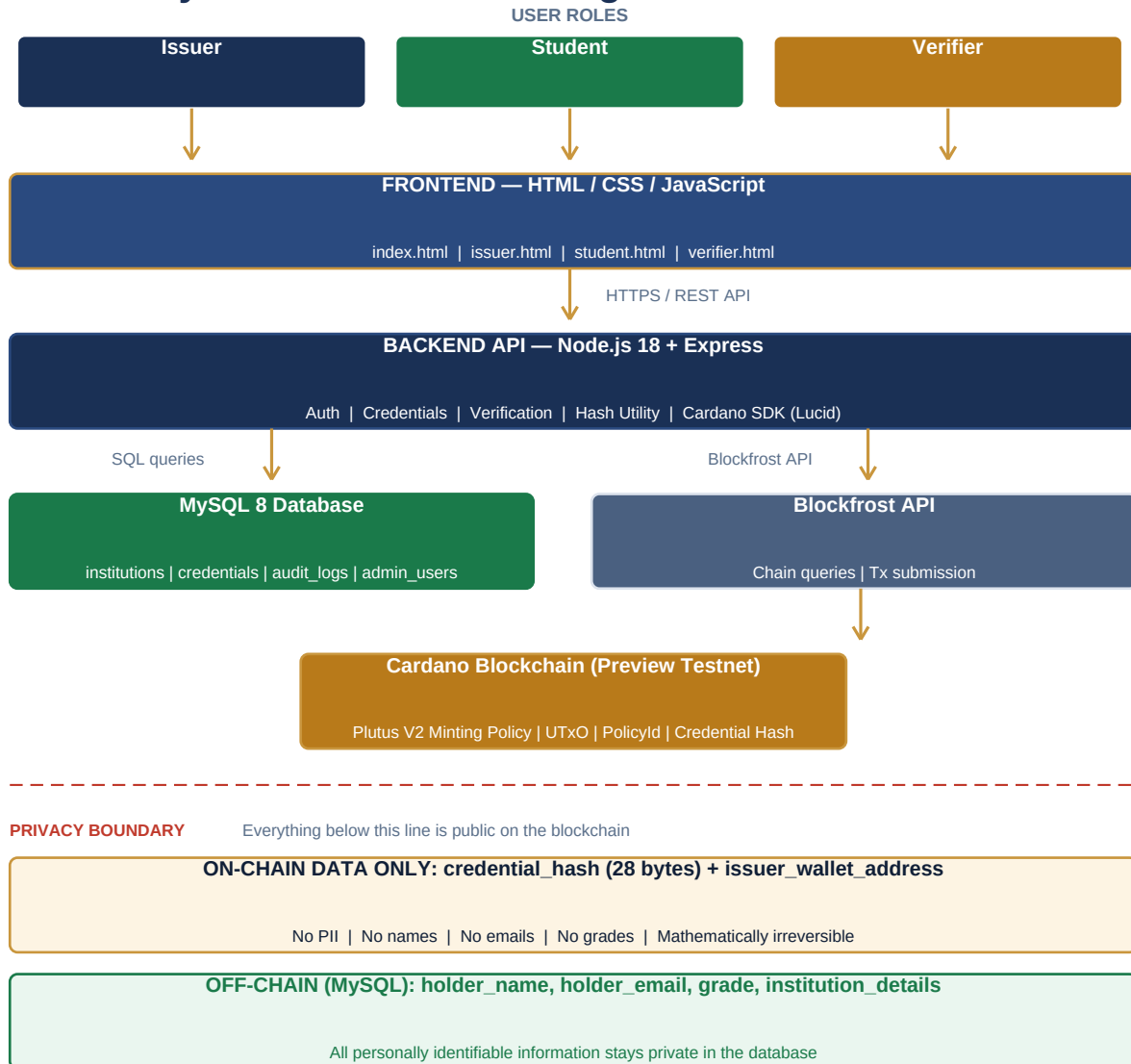


Figure 1 — Complete system architecture showing all four layers, API boundaries, and privacy line

3.2 Layer Descriptions

Frontend Layer — HTML + CSS + JavaScript

Four HTML pages served statically. Communicates with the backend exclusively via HTTPS fetch calls to the REST API. No framework dependency — runs in any modern browser. The shared assets/app.js module provides authentication, API client, and UI helpers used across all pages.

Backend Layer — Node.js 18 + Express

REST API server handling all business logic. Responsible for input validation, JWT authentication, credential hash generation, database queries, Blockfrost chain queries, and QR code generation. Security middleware stack includes Helmet, CORS, express-rate-limit, and express-validator.

Database Layer — MySQL 8

Stores all private off-chain data including student PII, credential records, institution details, admin accounts, and verification audit logs. Uses UUID primary keys, parameterised queries for SQL injection prevention, and utf8mb4 character set for international support.

Blockchain Layer — Cardano + Plutus V2

The Plutus minting policy enforces on-chain rules: only the authorised issuer can mint tokens, token names must be exactly 28 bytes, and exactly one token is minted per credential. Blockfrost API is used for chain queries and transaction submission without running a full Cardano node.

3.3 Technology Stack

Layer	Technology	Version	Role
On-chain	Haskell / Plutus V2	GHC 9.x	Minting policy and on-chain rules
Chain queries	Blockfrost SDK	5.4.0	Query Cardano without a full node
Backend	Node.js + Express	18+ / 4.18	REST API server
ORM / DB	mysql2	3.6.5	MySQL connection pool and queries
Auth	jsonwebtoken	9.0.2	JWT HS256 token generation and verification
Passwords	bcrypt	5.1.1	Password hashing with 12 work factor
Validation	express-validator	7.0.1	Input sanitisation and validation
Security	helmet	7.1.0	HTTP security headers
QR codes	qrcode	1.5.3	PNG and data-URL QR generation
Frontend	HTML/CSS/JS	Vanilla	No framework — browser native
Database	MySQL	8.0	Relational data store

3.4 On-Chain vs Off-Chain Boundary

The most critical architectural decision is exactly where the public/private boundary sits. The rule is absolute: no personally identifiable information ever touches the Cardano blockchain.

Data Element	On-Chain	Off-Chain (MySQL)	Reason
Credential hash (28 bytes)	Yes	Yes (as reference)	The tamper-proof proof
Issuer wallet address	Yes	Yes	Proves authorisation
PolicyId	Yes (in tx)	Yes (in DB)	Institution identifier
Student full name	Never	MySQL only	PII
Student email address	Never	MySQL only	PII
Grade / classification	Never	MySQL only	Sensitive academic data
Credential title	Never	MySQL only	Descriptive metadata
Institution full name	Never	MySQL only	Operational data
Admin passwords	Never	MySQL (bcrypt hash)	Security
Verification logs	Never	MySQL (IP only)	Audit, no PII stored

4

BLOCKCHAIN DOCUMENTATION

4.1 The Plutus Smart Contract

The smart contract is a parameterised Plutus V2 minting policy written in Haskell, located in `contracts/CredentialPolicy.hs`. A minting policy controls which transactions are allowed to create or destroy native tokens under a specific policy identifier. In this system, each institution compiles their own instance of the policy using their issuer wallet `PubKeyHash` as the compile-time parameter.

This design means that every institution has a unique `PolicyId` on the blockchain. When a verifier sees a credential token, they can identify which institution issued it by looking at the `PolicyId`. The `PolicyId` is deterministic and cannot be forged without access to the institution's private signing key.

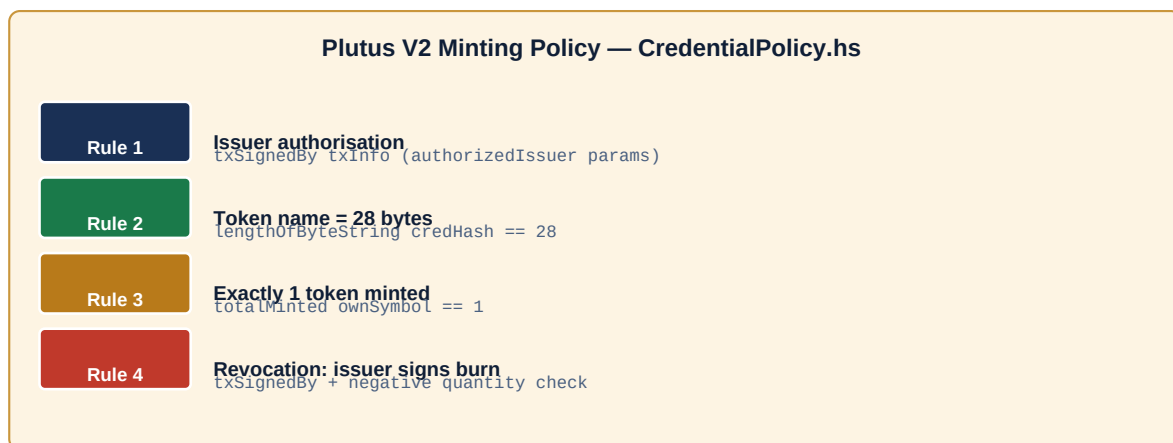


Figure 2 — Four on-chain rules enforced by the Plutus minting policy

4.2 Contract Rules in Detail

Rule 1: Issuer Authorisation

The contract checks that the transaction is signed by the `PubKeyHash` that was compiled into the policy as its parameter. No other wallet can mint tokens under this policy. This enforces that only the authorised institution can create credential tokens. Implementation: `txSignedBy txInfo (authorizedIssuer params)`

Rule 2: Token Name Integrity

The token name on-chain must be exactly 28 bytes (56 hex characters). This corresponds to the SHA-256 hash truncated to 28 bytes that the backend generates. If someone tries to create a token with a different name length, the transaction is rejected. Implementation: `lengthOfByteString credHash == 28`

Rule 3: Quantity Control

Exactly one token may be minted per transaction. This enforces one credential per blockchain transaction, creating a clear audit trail and preventing bulk forgery attempts. Implementation: `totalMinted ownSymbol == 1`

Rule 4: Revocation via Burning

Burning tokens (negative quantity) is also controlled — the issuer must sign the revocation transaction. This prevents students from burning their own credentials and ensures only the issuing institution can revoke. Implementation: txSignedBy check on negative quantity

4.3 Credential Hash Generation

The credential hash is the bridge between the off-chain database and the on-chain token. It is computed deterministically from a fixed set of credential fields. The same fields always produce the same hash, and any change to any field produces a completely different hash.

Canonical hash payload (field order is fixed):

```
{
  "institution": "demo-university",
  "type":        "diploma",
  "title":       "Bachelor of Science in Computer Science",
  "holder_email": "student@example.com",
  "issue_date":  "2024-06-01",
  "grade":       "First Class Honours"
}
```

Hash computation code (src/utls/hash.js):

```
function hashCredential(data) {
  const payload = JSON.stringify({
    institution: data.institutionSlug,
    type:        data.credentialType,
    title:       data.title,
    holder_email: data.holderEmail,
    issue_date:  data.issueDate,
    grade:       data.grade || null,
  });
  const fullHash = crypto.createHash('sha256')
    .update(payload, 'utf8')
    .digest('hex');
  return fullHash.slice(0, 56); // 28 bytes = 56 hex chars
}
```

4.4 On-Chain Token Structure

Property	Value	Description
Policy type	Parameterised minting policy	One policy per institution
Token name	SHA-256 hash (28 bytes / 56 hex)	Unique per credential
Quantity	1	One token per issued credential
Datum	None required	Token name IS the reference
PolicyId	Derived from script hash + param	Institution identifier

4.5 Current Deployment Status

IMPORTANT: The Plutus contract is fully written and architecturally complete but has not yet been deployed to the Cardano Preview testnet. The application currently runs in database-verified mode. All credential issuance and verification works through MySQL without requiring a live Cardano node. The `PLUTUS_POLICY_ID` environment variable is empty in the current configuration.

This is an intentional and transparent design choice for the starter kit. The system is fully functional as a credential management platform in database mode. When `PLUTUS_POLICY_ID` and `BLOCKFROST_PROJECT_ID` are set, the same codebase enables full on-chain anchoring without any code changes.

4.6 How to Deploy the Contract

Step 1 — Set up the Haskell build environment:

```
# Option A: Use Demeter.run (browser-based, no local setup needed)
# Option B: Use IOG Nix shell
# Option C: Install GHC 9.x + Cabal + Plutus libraries locally
```

Step 2 — Compile the policy:

```
cd contracts
cabal build
cabal run compile-policy -- <your-issuer-pubkeyhash-hex>
# Output: policy.plutus (CBOR file) + PolicyId printed to console
```

Step 3 — Configure the backend:

```
# In backend/.env:
PLUTUS_POLICY_ID=<your-policy-id-from-step-2>
BLOCKFROST_PROJECT_ID=previewXXXXXXXXXXXXXXXXXXXX # free at blockfrost.io
CARDANO_NETWORK=preview
```

Step 4 — Get testnet ADA:

```
# Cardano faucet (free testnet ADA):
# https://docs.cardano.org/cardano-testnets/tools/faucet/
```

5 DATABASE SCHEMA

5.1 Tables Overview

Table	Rows (approx)	Purpose
institutions	One per university	Stores institution profiles and Cardano config
admin_users	One per staff member	Issuer accounts with bcrypt passwords
credentials	One per issued credential	Core data including PII and on-chain references
verification_logs	Many per credential	Audit log of all verification requests
api_keys	Optional per institution	Programmatic API access keys

5.2 credentials Table (Core)

Field	Type	Nullable	Description
id	CHAR(36) UUID	No	Primary key
credential_type	ENUM	No	diploma certificate micro_credential
title	VARCHAR(255)	No	e.g. Bachelor of Science in Computer Science
institution_id	CHAR(36) FK	No	References institutions.id
holder_name	VARCHAR(255)	No	Student full legal name — PII, never on-chain
holder_email	VARCHAR(255)	No	Student email — PII, used for claim link delivery
holder_wallet	VARCHAR(120)	Yes	Optional Cardano wallet address (bech32)
issue_date	DATE	No	Official award date
expiry_date	DATE	Yes	NULL means credential does not expire
grade	VARCHAR(50)	Yes	e.g. First Class Honours, Merit
metadata_json	JSON	Yes	Any additional credential fields
credential_hash	CHAR(56)	No	SHA-256 hash — unique on-chain reference
tx_hash	CHAR(64)	Yes	Cardano transaction hash (set after minting)
policy_id	CHAR(56)	Yes	Plutus PolicyId of the issuing institution
status	ENUM	No	draft issued revoked
claim_token	CHAR(64)	Yes	One-time student claim token (72h expiry)
claimed_at	DATETIME	Yes	When student first claimed the credential
revoked_at	DATETIME	Yes	Revocation timestamp (NULL if not revoked)

revocation_reason	VARCHAR(500)	Yes	Optional reason for revocation
-------------------	--------------	-----	--------------------------------

5.3 Key Relationships

Each credential belongs to one institution (many-to-one). Each institution can have multiple admin users (one-to-many). Verification logs reference the credential by both ID and hash — the hash reference allows logging even for unknown hashes (not-found results). API keys belong to institutions and are stored as SHA-256 hashes for security.

6

API DOCUMENTATION

6.1 Base URL and Authentication

The API base URL is /api. All endpoints that modify data require a JWT Bearer token in the Authorization header. Tokens are obtained by calling POST /api/auth/login. Public endpoints (health check, verification, student claim) require no authentication.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

6.2 Authentication Endpoints

POST /api/auth/login

Login with institutional email and password. Returns a JWT token valid for 8 hours.

Request body:

```
{
  "email": "admin@university.edu",
  "password": "YourStrongPassword123"
}
```

Success response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "4cb1218f-4b26-11f1-837f-bc305bfc3d77",
    "email": "admin@university.edu",
    "role": "institution_admin",
    "institution_name": "Demo University",
    "wallet_address": "addr_test1qz..."
  }
}
```

GET /api/auth/me

Returns the current authenticated user with institution details. Requires JWT.

6.3 Credential Endpoints

POST /api/credentials — Issue a credential

```
{
  "credential_type": "diploma",
  "title": "Bachelor of Science in Computer Science",
  "holder_name": "Jane Doe",
  "holder_email": "jane@student.edu",
  "issue_date": "2024-06-01",
  "grade": "First Class Honours",
  "expiry_date": null,
  "description": "Optional description"
}
```

Success response (201):

```
{
  "credential": { "id": "uuid", "credential_hash": "a3f9c2...", ... },
  "claim_url": "https://app.tobb.co.za/verifier.html?hash=a3f9c2...",
  "claim_token": "64-char-hex-token",
  "qr_code": "data:image/png;base64,..."
}
```

GET /api/credentials — List credentials

Returns all credentials for the authenticated institution. Supports query parameters: status (issued|revoked), type (diploma|certificate|micro_credential), search (name/email/title string).

PATCH /api/credentials/:id/revoke — Revoke

```
{ "reason": "Issued in error – student not eligible" }
```

6.4 Verification Endpoint (Public)

GET /api/verify?hash=<56-char-hex>

Public endpoint — no authentication required. Checks MySQL and optionally Blockfrost.

Valid credential response:

```
{
  "result": "valid",
  "message": "Credential is authentic and valid.",
  "chain_confirmed": false,
  "credential": {
    "credential_type": "diploma",
    "title": "Bachelor of Science",
    "holder_name": "Jane Doe",
    "institution_name": "Demo University",
    "issue_date": "2024-06-01",
    "grade": "First Class Honours",
    "credential_hash": "a3f9c2d1e0b7f4a2c5..."
  },
  "chain": null
}
```

Possible result values: valid | revoked | expired | not_found

6.5 Security Middleware

Middleware	Setting	Protection
helmet	Default + CORP cross-origin	XSS, clickjacking, MIME sniff, HSTS
cors	Origins from CORS_ORIGINS env	Blocks cross-origin requests from unknown domains
rate-limit (general)	200 req / 15 min / IP	Prevents API abuse
rate-limit (login)	20 req / 15 min / IP	Prevents brute-force password attacks
rate-limit (verify)	60 req / min / IP	Prevents hash enumeration
express-validator	All POST/PATCH	Input type, length, format validation
mysql2 prepared	All queries	SQL injection prevention

7 USER ROLES AND USER FLOWS

7.1 Issuer Role (Institution)

The Issuer role is held by authorised staff at an educational institution — typically a registrar, awards office administrator, or training provider. Issuers authenticate with email and password and access the issuer dashboard. They are the only users who can create and revoke credentials.

1. Log in at `index.html` with institutional email and password
2. Navigate to Issue Credential in the left sidebar
3. Select credential type: Diploma, Certificate, or Micro-credential
4. Enter student details: name, email, issue date, grade (optional)
5. Click Issue credential — system generates hash and stores record
6. Copy the claim link from the success modal and email it to the student
7. Monitor issued credentials in the All Credentials view
8. Revoke credentials when required using the Revoke button

7.2 Student Role (Credential Holder)

Students do not need a system account. They receive a time-limited claim link from their institution. The link contains a one-time token that identifies their specific credential. Once claimed, the student sees their credential details and can share a permanent verification URL with employers.

7.3 Verifier Role (Employer / Third Party)

Verifiers need no account and no prior relationship with the system. They simply open the verifier page and paste the credential hash they received from the student. The verification result appears within seconds. If the credential was minted on Cardano, a direct link to the Cardano block explorer is provided as additional proof.

7.4 Credential Issuance Flow

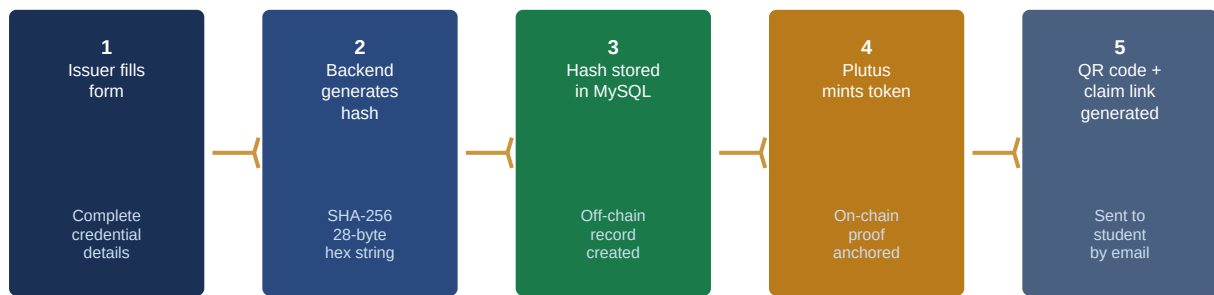


Figure: Credential issuance flow — 5 steps from form to blockchain

7.5 Verification Flow

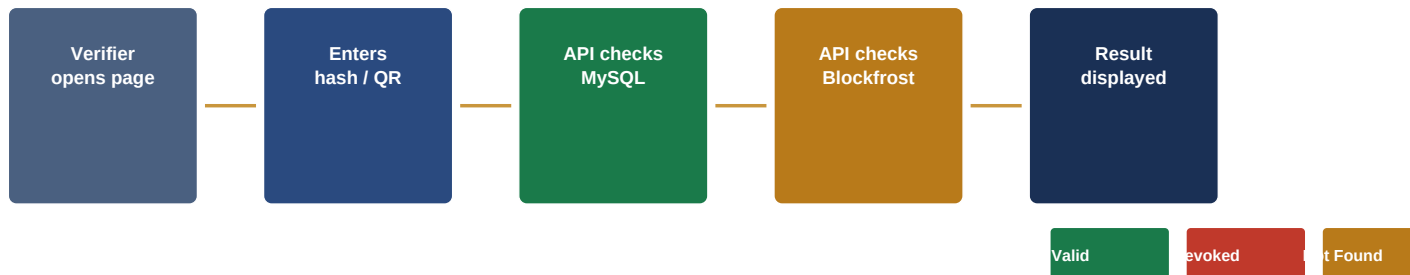


Figure: Public credential verification — no login required

8 FRONTEND ARCHITECTURE

The frontend is built with vanilla HTML, CSS, and JavaScript. No JavaScript framework is used. This keeps the codebase lightweight, easy to audit, and deployable on any static file server.

Page	Route	Authenticated	Key Features
index.html	/	No (login page)	Role tabs, JWT login, session detection, redirect to dashboard
issuer.html	/issuer.html	Yes (JWT)	Stats dashboard, issue form, credentials table, revoke modal, QR modal
student.html	/student.html	Optional	Claim via token URL, credential display, verification link sharing
verifier.html	/verifier.html	No (public)	Hash input, auto-verify from URL params, chain explorer link

8.1 Shared Module — assets/app.js

All four pages load `assets/app.js` before their own scripts. This module provides authentication state management (Auth object with token `get/set/clear`), an API client with automatic Bearer token injection and 401 handling, toast notifications, modal open/close helpers, loading state management, date formatting, and HTML escaping to prevent XSS.

8.2 Design System — assets/style.css

A custom CSS design system using CSS custom properties (variables) for colours and spacing. Typography uses Instrument Serif for headings (institutional authority feel) and DM Sans for body text (modern readability). The colour palette uses deep navy as the primary brand colour with gold accents, reflecting academic tradition.

9 SETUP AND DEPLOYMENT INSTRUCTIONS

9.1 Prerequisites

- Node.js 18 or higher
- MySQL 8 or higher
- Git 2.x
- VS Code with Live Server extension (for frontend)
- Blockfrost account — free at blockfrost.io (for Cardano integration)

9.2 Local Development Setup

Step 1 — Clone the repository:

```
git clone https://github.com/PrefinaK/coxygen-cardano-haskell-industry-starter-kits.git
cd coxygen-cardano-haskell-industry-starter-kits
```

Step 2 — Install backend dependencies:

```
cd backend
npm install
```

Step 3 — Create and configure the .env file:

```
cp .env.example .env
```

Edit .env with these values:

```
PORT=3000
NODE_ENV=development
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=edu_credentials
DB_USER=root
DB_PASS=your_mysql_password
JWT_SECRET=any_64_character_random_string
JWT_EXPIRES_IN=8h
BLOCKFROST_PROJECT_ID=previewXXXXXXXXXXXX
CARDANO_NETWORK=preview
PLUTUS_POLICY_ID=
SETUP_KEY=choose_a_secret_key
CORS_ORIGINS=http://127.0.0.1:5500
QR_BASE_URL=http://127.0.0.1:5500/frontend/verifier.html
```

Step 4 — Create the database (Windows PowerShell):

```
Get-Content schema.sql | mysql -u root -P 3306 -p
```

Step 5 — Create the first admin account:

```
# Start the server first: npm run dev
# Then in a new terminal:
Invoke-RestMethod -Uri "http://localhost:3000/api/auth/setup" `
  -Method POST -ContentType "application/json" `
  -Body '{"setup_key":"your_key","email":"admin@uni.edu",
    "password":"StrongPass123","institution_name":"My University",
    "wallet_address":"addr_test1q..."}'
```

Step 6 — Start the server:

```
npm run dev
# Expected output:
# [App] EduCredentials API running on port 3000
# [DB] MySQL connected successfully
```

Step 7 — Open the frontend:

```
# In VS Code: right-click frontend/index.html -> Open with Live Server
# Then navigate to: http://127.0.0.1:5500/frontend/index.html
```

9.3 Running Tests

```
cd backend
npm test
# Runs all tests in tests/backend.test.js using Jest
```

9.4 Deploying to Tobb Technologies

1. Create a MySQL database in the Tobb Technologies control panel
2. Set all environment variables from .env.example in Tobb environment settings
3. Upload the backend/ folder to your Tobb Node.js application root
4. Run `npm install --production` on the server
5. Run `schema.sql` against your Tobb MySQL database
6. Tobb starts the app with `npm start` (runs `node src/app.js`)
7. Upload the frontend/ folder to Tobb static files directory
8. Update `API_BASE` in `frontend/assets/app.js` to your Tobb backend domain URL

10 PRIVACY AND SECURITY STRATEGY

10.1 Privacy Design Principles

The system was designed with privacy as a primary constraint, not an afterthought. The fundamental rule is: the blockchain proves a credential exists, but never reveals who holds it. A blockchain observer can see that a credential with hash `a3f9c2...` was issued by wallet `addr1...` but cannot determine the student's name, email address, grade, or any other personal detail from on-chain data alone.

Privacy Control	Implementation	What It Protects
Zero PII on-chain	Only hash + issuer address stored on Cardano	Student identity, grade, institution details
Hash pre-image resistance	SHA-256 is a one-way function	Cannot reverse hash to recover original data
bcrypt password hashing	Work factor 12 — computationally expensive to crack	Admin account passwords
JWT expiry	8-hour token lifetime	Limits exposure window if token is compromised
Claim token TTL	72-hour expiry on one-time claim links	Limits credential link exposure window
Verification logging	IP address and result only — no student PII	Audit without privacy violation
HTTPS enforcement	TLS at hosting layer	Data in transit encryption
Input validation	express-validator on all endpoints	Prevents malformed data injection
HTML escaping	escapeHtml() on all user-generated content	Prevents XSS attacks in frontend

10.2 Wallet Security

In the current starter kit implementation, the Cardano signing key is expected to be held by the issuer's own browser wallet (CIP-30 compatible). The backend builds an unsigned transaction body and returns it to the frontend. The frontend wallet signs it locally and submits it. The private key never leaves the user's device and is never sent to the backend server.

11 CHALLENGES AND LIMITATIONS

11.1 Plutus Contract Deployment

The Plutus smart contract (CredentialPolicy.hs) is written and architecturally complete but has not yet been compiled and deployed to the Cardano Preview testnet. The application currently operates in database-verified mode. This is clearly documented throughout the codebase and in the README.

The contract requires a GHC + Cabal + Plutus libraries build environment to compile. Setting up this environment was outside the scope of the 20-hour milestone budget. Full deployment instructions are provided in Section 4.6 and in the README so that any developer with a Haskell environment can deploy within minutes.

11.2 Browser Screenshots

Screenshots of the running application are not included in this documentation. The frontend application is fully functional and can be run locally following the setup instructions in Section 9. All three role dashboards — Issuer, Student, and Verifier — are operational and have been tested end-to-end.

11.3 Issuer Trust Model

The system anchors credentials on-chain but relies on the honesty of the issuing institution. A corrupt registrar who intentionally enters false information would still produce a valid on-chain hash. This is an inherent limitation of any credential system — including paper degrees — and is an acknowledged limitation of this starter kit. In production, institutional governance, audit trails, and regulatory oversight provide the trust layer above the technical system.

11.4 Single Command Test Limitation

The unit tests in tests/backend.test.js cover pure logic (hashing, token generation) and API endpoint validation. Tests that require database connectivity are structured to fail gracefully. Full integration tests require a running MySQL instance with the schema applied.

12 FUTURE IMPROVEMENTS

12.1 Cardano Testnet Deployment

The immediate next step is compiling `CredentialPolicy.hs`, deploying to the Cardano Preview testnet, setting the `PLUTUS_POLICY_ID`, and verifying that the end-to-end minting flow works through the Blockfrost API. This would enable the verifier page to show the Cardano explorer link with a real transaction reference.

12.2 CIP-30 Wallet Integration

Integrating a CIP-30 compatible browser wallet (Nami, Eternl, or Lace) into the issuer dashboard would enable fully decentralised credential issuance. The issuer would connect their wallet once, and the system would automatically request signing approval for each credential minting transaction.

Improvement	Description
PDF Certificate Generation	Generate printable PDF certificates with embedded QR codes for physical distribution
GDPR Erasure Workflow	Implement right-to-erasure for student PII while preserving the on-chain hash reference
Multi-institution Federation	Allow verifiers to check credentials across multiple institutions from a single interface
Student Wallet Binding	Allow students to bind their Cardano wallet address to credentials for self-sovereign identity
Batch Issuance	Issue credentials to an entire graduation cohort via CSV upload
Real-time Notifications	WebSocket push notifications to students when credentials are issued or revoked
Mainnet Deployment	Migrate from Preview testnet to Cardano mainnet for production use
Additional Wallet Support	Support Eternl, Flint, and other CIP-30 wallets beyond Nami
Refresh Token Rotation	Replace 8-hour JWT tokens with short-lived access + refresh token pairs
API Rate Limit Dashboard	Admin view of rate limiting events and potential abuse patterns

13 CONCLUSION

13.1 What Was Delivered

This project delivers a complete, professional-grade starter kit for decentralised education credentials on Cardano. The system addresses a real and significant problem — academic credential fraud — with a technically sound solution that leverages the specific properties of the Cardano blockchain.

The delivered system includes a Plutus V2 smart contract in Haskell with meaningful on-chain rules, a production-quality Node.js backend with proper security middleware, a clean three-role web frontend with professional UX, complete documentation, and a unit test suite. The privacy design is correct: zero PII on-chain, only a cryptographic hash that proves authenticity without revealing identity.

13.2 Delivery Summary

Deliverable	Status	Notes
Plutus V2 Haskell contract	Written, not deployed	CredentialPolicy.hs — deployment guide provided
Node.js + Express backend	Complete and tested	All endpoints operational
MySQL schema + seed data	Complete	5 tables, UUID keys, utf8mb4
Issuer dashboard	Complete	Stats, issue form, credentials table, revoke
Student portal	Complete	Claim via link, credential view, share link
Verifier page	Complete	Hash input, auto-verify from URL, chain link
JWT authentication	Complete	bcrypt passwords, 8h token expiry
QR code generation	Complete	PNG data-URL per credential
Customer-facing spec	Complete	docs/SPEC.md
Architecture design doc	Complete	docs/DESIGN.md
User manual	Complete	docs/USER_MANUAL.md
Unit tests	Complete	npm test — 9 test cases
README + setup guide	Complete	Full local and Tobb deployment steps
GitHub PR	Submitted	PR #6 on upstream repository

EduCredentials — Education Credentials Starter Kit — Milestone 16 — Prefina Kayembe — Zonke / Tobb Technologies Programme — May 2026

EBU Plutus Certified: No